

Modelos Lineares para Classificação

Alberto Paccanaro
EMap – FGV

www.paccanarolab.org

O material e as imagens nestes slides são de (ou adaptados de):
C. Bishop, Reconhecimento de Padrões e Aprendizado de Máquina, Springer, 2006

O problema de classificação

Dado um vetor de entrada $\mathbf{x}$, atribuí-lo a uma das K classes discretas C_k onde $k = 1, \dots, K$.

As classes são disjuntas, o espaço de entrada é dividido em *regiões de decisão*

Limites: *fronteiras de decisão* (*decision boundaries, decision surfaces*).

Modelos lineares: as fronteiras de decisão são funções lineares

hiperplano $(D-1)$ -dimensional no espaço de entrada D dimensional.

Linearmente separáveis: conjuntos de dados cujas classes podem ser separadas exatamente por superfícies de decisão lineares

Representando valores alvo

Duas classes ($K = 2$)

- uma variável, representação binária $t \in \{0, 1\}$
 $t = 1$ representa a classe C_1
 $t = 0$ representa a classe C_2 .
- *valor de t é a probabilidade de que a classe seja C_1 .*

$K > 2$ classes

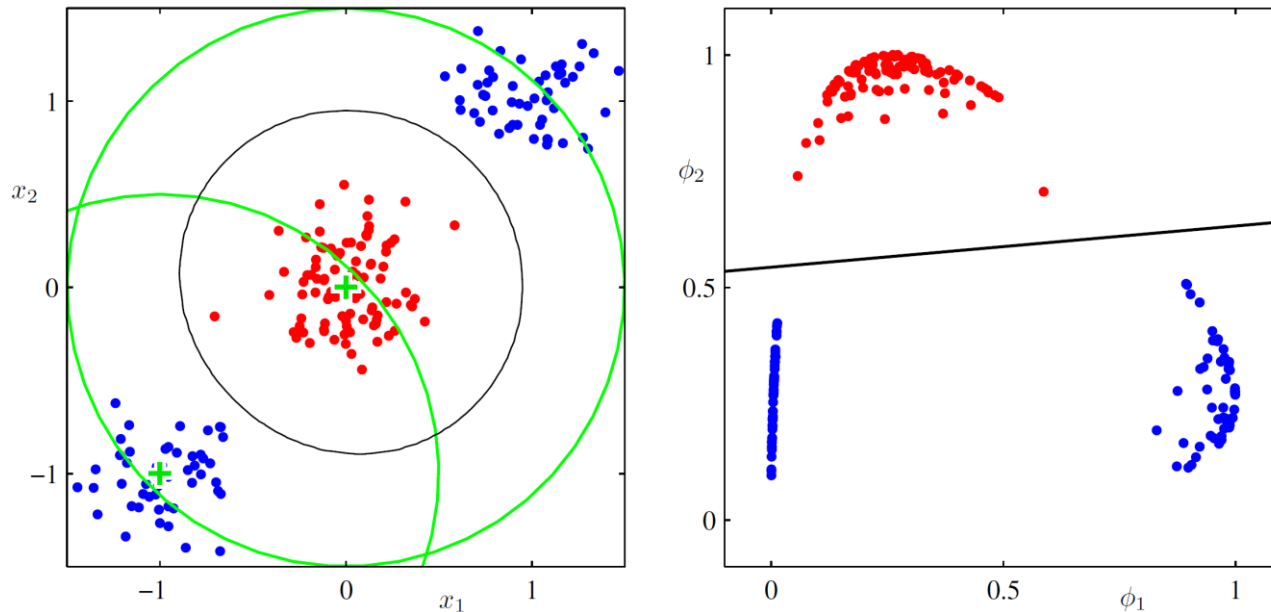
- é conveniente usar um esquema de codificação 1-de- K

$$\mathbf{t} = (0, 1, 0, 0, 0)^T$$

Nota sobre basis functions

Podemos primeiro aplicar uma transformação não linear das entradas $\phi(x)$ – *assim como fizemos na regressão linear*.

O problema de classificação pode se tornar mais fácil



Como falamos anteriormente, existem 3 abordagens

~~1. Funções discriminantes~~

2. **Modelos Gerativos Probabilísticos:** modelar $p(\mathbf{x} | C_k)$ e $P(C_k)$ e então usar o teorema de Bayes para inferir $p(C_k | \mathbf{x})$ (a *abordagem generativa*)

3. **Modelos Discriminativos Probabilísticos:** modelar $p(C_k | \mathbf{x})$ diretamente

2. Modelos Gerativos Probabilísticos

Modelos com limites de decisão lineares surgem de suposições simples sobre a distribuição dos dados...

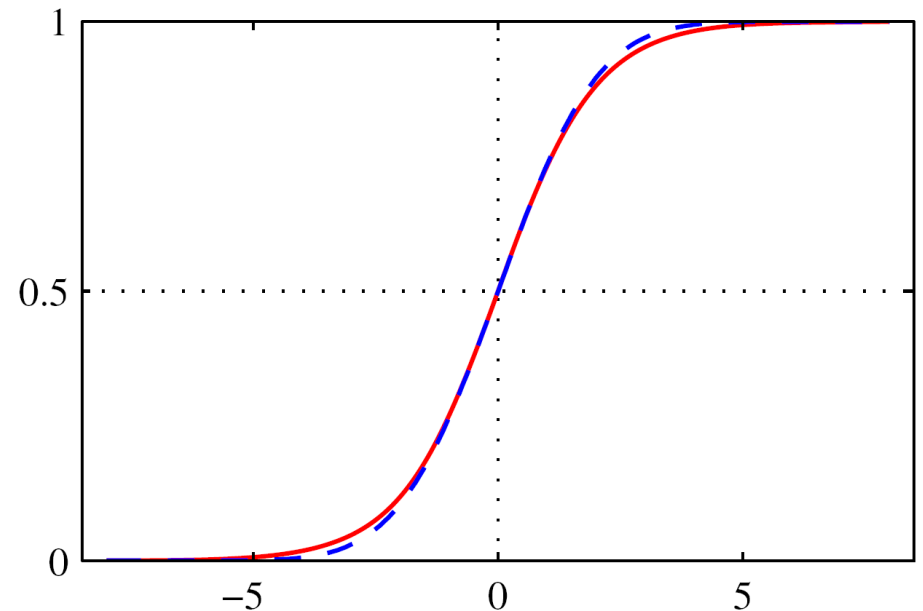
Duas classes

$$\begin{aligned} p(\mathcal{C}_1|\mathbf{x}) &= \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} \\ &= \frac{1}{1 + \exp(-a)} = \sigma(a) \end{aligned}$$

onde definimos:

$$a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)}$$

função sigmoide



K > 2 classes

$$\begin{aligned} p(C_k|\mathbf{x}) &= \frac{p(\mathbf{x}|C_k)p(C_k)}{\sum_j p(\mathbf{x}|C_j)p(C_j)} \\ &= \frac{\exp(a_k)}{\sum_j \exp(a_j)} \end{aligned} \quad \text{função softmax}$$

onde definimos:

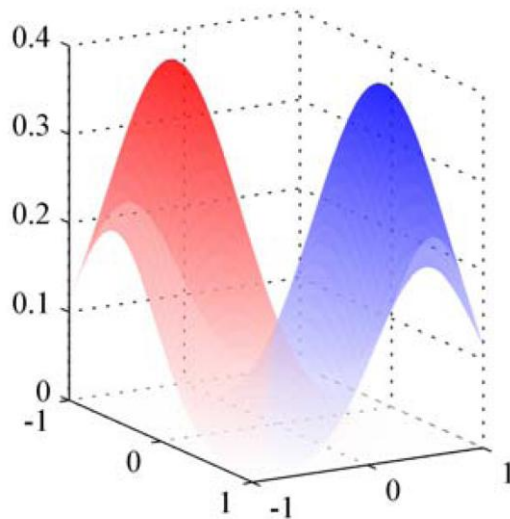
$$a_k = \ln p(\mathbf{x}|C_k)p(C_k)$$

PERGUNTA

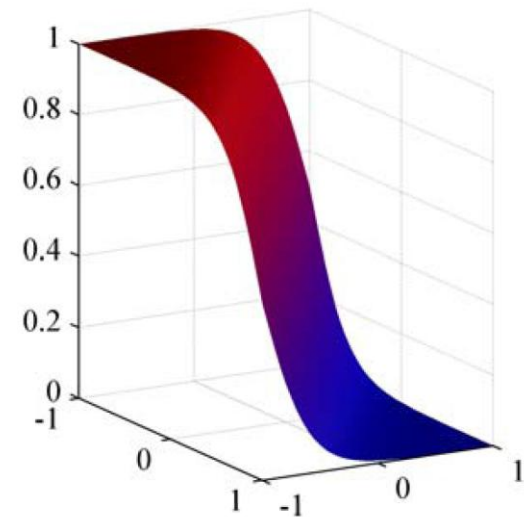
Como a forma para as densidades condicionais de classe $p(\mathbf{x}|C_k)$ afeta a forma de $p(C_k|\mathbf{x})$?

2 classes, features contínuas, densidades condicionais de classe gaussianas, mesma covariância

Obtemos uma função linear de $\mathbf{x}$ no argumento do sigmóide logístico.



$p(\mathbf{x}|C_k)$



$p(C_1|\mathbf{x})$

K classes, features contínuas, densidades condicionais de classe gaussianas, mesma covariância

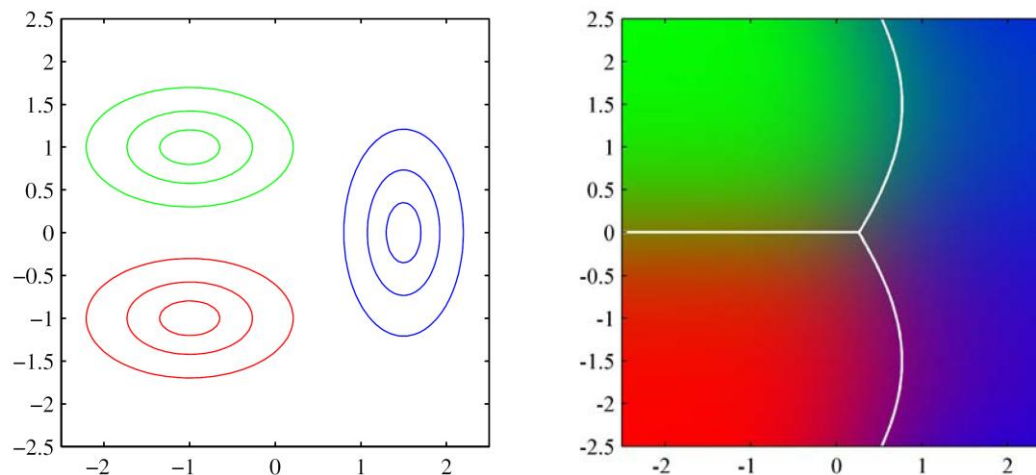
Os $a_k(\mathbf{x})$ são funções lineares de $\mathbf{x}$

Como antes, os limites de decisão serão definidos por funções lineares de $\mathbf{x}$.

Novamente temos um modelo linear generalizado.

K classes, features contínuas, densidades condicionais de classe gaussianas, covariância diferente

obtemos funções *quadráticas de x*, dando origem a um discriminante quadrático.



Esses resultados podem ser estendidos para as densidades condicionais de classe da família exponencial de distribuições.

3. Modelar $p(C_k | \mathbf{x})$ diretamente – *Modelos Discriminativos Probabilísticos*

A probabilidade posterior da classe C_1 pode ser escrita como um sigmóide logístico (ou softmax para multiclasse) atuando em uma função linear de $\mathbf{x}$.

Idea: usar a forma funcional do modelo linear generalizado e determinar seus parâmetros diretamente usando a máxima verossimilhança.

Ou seja, maximizamos uma função de verossimilhança definida por $p(C_k | \mathbf{x})$ (treinamento discriminativo).

O modelo geral

$$y(\mathbf{x}) = f(\mathbf{w}^T \mathbf{x} + w_0)$$

Para prever rótulos de classe, transformamos a função linear de w usando uma *função de ativação não linear* $f(\cdot)$

- O **modelo não é linear** nos parâmetros (lembre-se que a regressão linear foi linear nos parâmetros)

- As **superfícies de decisão são funções lineares de x** , mesmo que a função $f(\cdot)$ é não linear
(corresponde a $\mathbf{w}^T \mathbf{x} + w_0 = \text{constante}$)

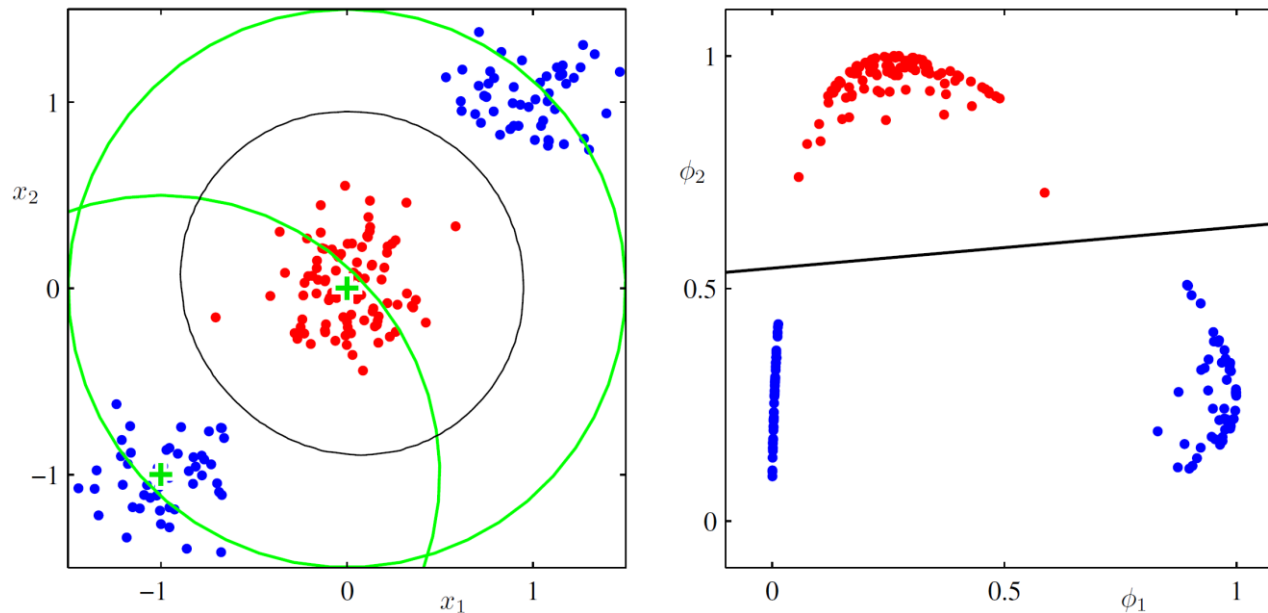
Modelos lineares generalizados

... apenas um lembrete...

basis functions (fixas)

Os algoritmos são igualmente aplicáveis se primeiro aplicarmos uma transformação não linear das entradas $\phi(x)$

Os limites de decisão são lineares no espaço de features $\phi(x)$ e são não lineares no espaço original



Regressão Logística (2 classes)

$$p(\mathcal{C}_1|\phi) = y(\phi) = \sigma(\mathbf{w}^T \phi)$$
$$p(\mathcal{C}_2|\phi) = 1 - p(\mathcal{C}_1|\phi)$$

Um sigmóide logístico atuando em uma função linear do vetor de features ϕ

Na terminologia das estatísticas, isso é conhecido como regressão logística (ou Discriminante Logístico ou Discriminação Logística)

Este é um modelo para classificação, não para regressão 😊

Apenas M parâmetros.

Para aprender o vetor $\mathbf{w}$ precisamos de uma Função de Erro para minimizar... (asi como fizemos para a Regressão Linear)

Vamos usar a máxima verossimilhança para determinar os parâmetros $\mathbf{w}$ do modelo.

Conjunto de N datapoints: $\{\phi_n, t_n\}$

$$\begin{aligned}\phi_n &= \phi(\mathbf{x}_n) \\ t_n &\in \{0, 1\}\end{aligned}$$

Função de verossimilhança:

$$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^N y_n^{t_n} \{1 - y_n\}^{1-t_n} \quad \mathbf{t} = (t_1, \dots, t_N)^T$$
$$y_n = p(\mathcal{C}_1|\phi_n)$$

Pegue o logaritmo negativo:

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

$$y_n = \sigma(a_n) \quad a_n = \mathbf{w}^T \phi_n$$

**Função de erro:
Cross Entropy**

IMPORTANTE

função de erro de entropia cruzada surge da maximização da probabilidade dos dados em problemas de classificação binária

... da mesma forma que...

a função de erro de soma de quadrados surge da maximização da probabilidade dos dados sob uma suposição de ruído gaussiano.

A derivada da função de erro CE com função de ativação σ

$$a_n = \mathbf{w}^T \phi_n$$

$$y_n = \sigma(a_n)$$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n$$

A contribuição para o gradiente do datapoint n é dada pelo 'erro' $(y_n - t_n)$ entre o valor alvo e a previsão do modelo, vezes a entrada para o datapoint n .

Tente você mesmo!

use o fato de que: $\frac{\partial \sigma(a)}{\partial a} = \sigma(a) \cdot (1 - \sigma(a))$

Observação importante: esta é a mesma forma que o gradiente da função de erro SSE para o modelo de regressão linear...

Linear Reg. com SSE

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2$$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N \{\mathbf{w}^T \phi(\mathbf{x}_n) - t_n\} \phi(\mathbf{x}_n)^T$$

Logística Reg. com CE

$$E(\mathbf{w}) = - \sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n$$

Mas não há fórmula fechada desta vez... por quê?

Para aprender $\mathbf{w}$ pode-se usar um procedimento sequencial onde os datapoints são apresentados um de cada vez e os vetores de peso são atualizados pela descida do gradiente estocástico

Mínimos quadrados iterativos reponderados (IRLS)

A atualização de Newton-Raphson:

$$\mathbf{w}^{(\text{new})} = \mathbf{w}^{(\text{old})} - \mathbf{H}^{-1} \nabla E(\mathbf{w})$$

Aplicando-o à função de erro de entropia cruzada para o modelo de regressão logística:

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n = \mathbf{\Phi}^T (\mathbf{y} - \mathbf{t})$$

$$\mathbf{H} = \nabla \nabla E(\mathbf{w}) = \sum_{n=1}^N y_n (1 - y_n) \phi_n \phi_n^T = \mathbf{\Phi}^T \mathbf{R} \mathbf{\Phi}$$

onde $\mathbf{R}$ é uma matriz diagonal $N \times N$ com elementos

$$R_{nn} = y_n(1 - y_n)$$

Algoritmo iterativo, com fórmula de atualização:

$$\mathbf{w}^{(\text{new})} = (\Phi^T \mathbf{R} \Phi)^{-1} \Phi^T \mathbf{R} \mathbf{z}$$

Onde $\mathbf{z} = \Phi \mathbf{w}^{(\text{old})} - \mathbf{R}^{-1}(\mathbf{y} - \mathbf{t})$

Regressão Logística Multiclasse

$$a_k = \mathbf{w}_k^T \phi \quad p(\mathcal{C}_k | \phi) = y_k(\phi) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$$

$$p(\mathbf{T} | \mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{n=1}^N \prod_{k=1}^K p(\mathcal{C}_k | \phi_n)^{t_{nk}} = \prod_{n=1}^N \prod_{k=1}^K y_{nk}^{t_{nk}}$$

$$E(\mathbf{w}_1, \dots, \mathbf{w}_K) = -\ln p(\mathbf{T} | \mathbf{w}_1, \dots, \mathbf{w}_K) = -\sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}$$

Função de erro de entropia cruzada multiclasse

$$\nabla_{\mathbf{w}_j} E(\mathbf{w}_1, \dots, \mathbf{w}_K) = \sum_{n=1}^N (y_{nj} - t_{nj}) \phi_n$$

$\mathbf{T}$ (N×K)

Gradiente para
atualização iterativa

(A extensão do IRLS para multiclasse também é possível.)

IMPORTANTE

A derivada da função de erro da Soma dos Quadrados (SSE) para um modelo de regressão linear assume a forma de:

'erro' ($y_n - t_n$) vezes o vetor de features ϕ_n

... de forma similar...

para função de ativação sigmóide logística e função de erro de entropia cruzada &

para função de ativação softmax e função de erro de entropia cruzada multiclasse

... novamente obtemos esta mesma forma simples!